Federated Learning for Distributed IIoT Intrusion Detection Using Transfer Approaches

Jiazhen Zhang, Chunbo Luo, *Member, IEEE*, Marcus Carpenter, and Geyong Min, *Member, IEEE*

***Abstract*—Considering that low-cost and resource-constrained sensors coupled inherently could be vulnerable to growing numbers of intrusion threats, industrial Internet-of-Things (IIoT) systems are faced with severe security concerns. Data sharing for building high-performance intrusion detection models is also prohibited due to the sensitivity, privacy, and high value of IIoT data. This article presents an anomaly-based intrusion detection system with federated learning for privacy-preserving machine learning in future IIoT networks. To tackle the urgent issue of training local models with non-independent and identically distributed (non-IID) data, we adopt instance-based transfer learning at local. Furthermore, to boost the performance of this system for IIoT intrusion detection, we propose a rank aggregation algorithm with a weighted voting approach. The proposed system achieves superior detection performance with 95.97% and 73.70% accuracy for AdaBoost and Random Forest, respectively, outperforming the baseline models by 12.72% and 14.8%.**

***Index Terms*—Federated learning, industrial IoT (IIoT), network intrusion detection, transfer learning.**

I. Introduction

As the foundation of Industry 4.0, the industrial Internet of Things (IIoT) acts as the platform for connecting humans and edge devices with the Internet. The IIoT aims to connect real-world scenarios with distributed computing principles to achieve smart manufacturing, resource management, and other processes such as real-time remote healthcare [1]. However, when massive amounts of data are collected, transmitted, and processed on the IIoT network, preserving data privacy and system security become top concerns for IIoT adoption [2], [3].

Compared with the traditional network hardware, IoT and IIoT devices usually have low manufacturing costs and less maintenance, making them vulnerable to attacks [4]. Limited device self-defence capabilities result in frequent attacks targeting the IoT and IIoT devices, which also cause severe damage. In 2020, up to 5.7 billion attack attempts were reported from exposed IoT devices. Most of them were distributed denial-of-service (DDoS) and Botnet attacks, launched by Mirai, Gafgyt, and Mozi malware families [5]. It is crucial to identify threats as well as unauthorized access for resource-constrained devices facing an increase in IoT network intrusions. A common strategy to detect intrusions and network layer attacks is to use an intrusion detection system (IDS).

An IDS monitors network activities and analyzes received packets to identify threats from network traffic flow. Signature- and anomaly-based are two main detection strategies for IDS. Signature-based IDS, often known as rule-based IDS, identifies threats by comparing attack rules or patterns stored in signature libraries [6]. This detection strategy is effective for known attacks with a high detection rate and is suitable for resource limited devices [7]. However, relying on preset rules or attack signatures means that this method lacks capabilities for zero-day attacks or any unrecorded intrusions. Continuous system maintenance would be essential to guarantee its performance.

Anomaly-based IDS identifies attacks by classifying normal samples and malicious activities, including intrusions and permission abuses in the network traffic flow. This method can achieve a higher detection rate for unknown threats than signature-based IDS. However, high false-alarm rate issues still need to be resolved for anomaly-based intrusion detection [6]. Recent works in anomaly-based IDS adopt machine learning (ML) approaches to enhance model performance [8]. Some works used deep learning approaches for anomaly detection tasks, which significantly improved anomaly detection accuracy [2], [9]. Nonetheless, the neural network interpretability issues and massive model training requirements restrict their applications in IIoT intrusion detection scenarios.

To deploy ML-based IDS for edge devices, there are two main ways: 1) distributed; and 2) centralized. Distributed IDS works on the device to process training and testing locally to ensure data privacy and model personalization. However, training data collected at local for IIoT devices could be unbalanced, impacting model performance negatively [10]. The centralized strategy offloads IDS to the high-performance cloud servers and provides IDS services via the Internet [1]. This strategy relies on stable connections between devices and the server and can be invalid once the links are blocked or jammed. Attacks such as DDoS can exhaust all the bandwidth for a victim device,

Manuscript received 28 March 2022; revised 25 April 2022, 19 May 2022, 21 August 2022, and 29 September 2022; accepted 10 October 2022. Date of publication 25 October 2022; date of current version 20 June 2023. The work of Jiazhen Zhang was supported by the China Scholarship Council Studentship and the University of Exeter under Grant 202008060358. The work of Marcus Carpenter was supported by the EPSRC iCASE with British Telecommunications Plc, under Grant 2237757. This work was supported by the Horizon 2020 Framework Program under Grant 101008297. Paper no. TII-22-1294. *(Corresponding authors: Chunbo Luo; Geyong Min.)*

The authors are with the Department of Computer Science, University of Exeter, EX4 4QF Exeter, U.K. (e-mail: jz454@exeter.ac.uk; c.luo@exeter.ac.uk; mc844@exeter.ac.uk; g.min@exeter.ac.uk).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TII.2022.3216575.

Digital Object Identifier 10.1109/TII.2022.3216575

1551-3203 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 08:06:50 UTC from IEEE Xplore. Restrictions apply.

and then, seize control as the anomaly detection services are disconnected.

Here, we summarize the key challenges for anomaly-based network intrusion detection in IIoT scenarios, which this article aims to address. First, data processed in the distributed devices are usually privacy-preserved [11]. Second, for federated ML, localized datasets in different nodes are usually non-independent and identically distributed (non-IID), which affects the performance of coordination between devices. Third, most IoT/IIoT devices are resource-constrained [9], while most existing works in federated learning IDS [2], [9], [11] adopt deep neural networks, which may be too heavy to deploy as an on-device IDS.

Our work provides a novel approach by designing lightweight ML algorithms for anomaly-based IDS into the federated learning architecture of IIoT to tackle the above challenges. This approach adopts the ensemble ML as base learners and deploys the experiment on a physical testbed to simulate a physical IoT/IIoT intrusion detection scenario. Furthermore, we propose a two-step aggregation boost strategy and use one comprehensive model to satisfy all distributed clients to improve the model aggregation performance. Maximum mean discrepancy (MMD) is introduced for evaluating data distributions to set weights with well-tuned models as ensemble weighted voting for final predictions. This strategy can optimize the usage of all updated models in aggregation rather than simply averaging all parameters like FedAvg [12]. Furthermore, to address non-IID data distribution and data class-imbalanced issues, we adopt an instance-based transfer learning algorithm [13] to reuse labelled data and assist the localized model training process. We design an adequate experiment to compare the accuracy with or without data transfer to show the significant improvement when using the same ML models locally. To the best of our knowledge, this is the first attempt to propose lightweight federated learning with weighted voting aggregation technique in IoT/IIoT security and introduce instanced-based transfer learning to address non-IID data distribution and class-imbalanced issues in the IoT/IIoT network scenario.

In summary, our main contributions and innovations are as follows.

1) Introducing a novel framework of anomaly-based IDS for resource-constraint IoT/IIoT devices adopting the privacy-preserved ML mechanism.
2) Proposing an instance-based transfer learning approach on devices to address non-IID data distribution and class imbalanced issues for localized training data.
3) Designing a two-step aggregation method in federated learning using ensemble model voting techniques with weights evaluated by the MMD to enhance overall prediction performance.
4) The proposed framework achieves excellent results in extensive experiments using real-world intrusion detection data. The experiment is evaluated on a hardware test platform to simulate real-world IoT/IIoT scenarios. The results confirm that the instance-based transfer learning and the weighted voting aggregation can significantly improve the accuracy.

TABLE I
DESCRIPTIONS OF ALL NOTATIONS

Notations	Descriptions
D^S	Set of the public training data (source domain).
$D^T_{(\cdot)}$	Set of the localized training data (target domain).
D_{test}	Set of the test data.
$\Gamma_{(\cdot)}$	Lightweight ML models in the proposed system.
W^S	Set of weights for samples in the source.
W^T	Set of weights for samples in the target.
H	Set of the predicted labels.
$d_{(\cdot)}$	Set of binary data for single attack type.
$l(\cdot)$	Base learner in ensemble learning models.
$\gamma_{(\cdot)}$	Attack classifiers from rank aggregation.
$\text{pred}(\cdot)$	Prediction scores for test data.

The rest of this article is organized as follows. Section II reviews recent work in three topics: 1) IoT/IIoT network intrusion detection; 2) federated learning; and 3) transfer learning. Section III proposes the framework of federated transfer learning design and key features. Section IV and Section V present the main experiment design and the results. In Section VI, we present further discussions, limitations, and future research directions for this work. Finally, Section VII concludes this article.

II. Related Work

A. Anomaly-Based Intrusion Detection in IoT/IIoT Security

The IIoT is a special form of IoT, applied with specific settings to monitor, collect, analyze, and manage manufacturing processes in industrial scenarios [14]. Compared with IoT, there are some specific concerns for IIoT with regards to device communication and management and industrial control systems (ICS) [15]. ICS has a closer relationship with the physical world, aiming to manage industrial processes using several types of the control system and associated components [16]. Caselli et al. [17] proposed a sequence-aware approach for ICS sequence attack detection by identifying patterns of network events and extracting semantic meanings. Connecting with the ICS and managing industries such as intelligent factories or grids [14], data stored on the IIoT devices are usually confidential and directly related to real-world manufacturing environments, making the IIoT network security crucial.

Some previous work in IoT/IIoT has proved deploying IDS is an effective way to identify threats and protect security of the network [16], [18], [19]. As introduced above, anomaly-based IDS can be more effective for classifying unknown threats compared with the signature-based schemes. Learning-based methods can provide promising solutions to improve the overall detection rate for intrusion detection systems [6]. Bostani et al. [19] proposed a hybrid system of anomaly-based and specification-based IDS using an unsupervised optimum path forest approach in a 6LoWPAN network. This system used the MapReduce framework to connect and manage multiple nodes and detect selective-forwarding and sinkhole attacks. Eskandari et al. [18] developed an anomaly-based intrusion detection platform to detect varieties of attacks, including DoS attacks and brute-force attacks targeting IoT devices. The platform used

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 08:06:50 UTC from IEEE Xplore. Restrictions apply.

supervised ML techniques to detect outliers from the network traffic flow. This work adopted gateways as a localized server to receive and process data from IoT end devices.

Supervised ML has improved anomaly-based detection, but it relies on adequate training data and high-performance hardware. Localized data without proper processing may not suit model training, and data from different devices may have independent distributions. Nguyen et al. [20] proposed a poisoning attack targeting the intrusion detection model itself. This method can "backdoor" the detection model by gradually injecting a small amount of malicious traffic into the benign traffic so that the detection model does not identify this as anomalous and sees the malicious data as benign. Sharing data between multiple IoT/IIoT devices can improve the robustness and performance of local IDS as some threats may be unknown at local but are already identified in other devices. However, network traffic data are usually sensitive and private, which prohibits data sharing in the IoT/IIoT network. To mitigate the constraints mentioned above, we propose a federated learning framework for knowledge sharing between IIoT devices with instance-based transfer learning to assist model training at local, which reuses data from public datasets to boost the IDS performance.

B. Federated Learning

Federated learning is designed for training models on IoT or mobile devices to address the data island dilemma without violating data privacy restrictions [12]. Yang et al. [21] categorized federated learning into three different settings: 1) sample-based federated learning, which is when local data in different devices share the same features but in a different sample space; 2) feature-based federated learning, which is when the clients hold the same sample space but with different feature space; and 3) federated transfer learning, which is using transfer learning approaches to aggregate model updates or localize the public model for the scenario when neither sample space nor feature space is shared between clients.

Some recent works in anomaly-based IDS using federated learning are listed as follows. Rahman et al. [10] compared centralized and on-devices intrusion detection with federated learning intrusion detection for IoT networks. The federated learning method can coordinate localized model training on multiple devices by aggregating model updates. Nguyen et al. [9] proposed DIoT-IDS for anomaly detection in IoT devices using federated self-learning. DIoT-IDS used recurrent neural networks (NNs) on a local security gateway to process network traffic flow and avoid the hardware limitations for IoT devices. However, this system relies on localized data, making it hard to detect unknown attacks. Fan et al. [2] proposed the IoTDefender using federated transfer learning for 5G-enabled IoT networks where a multiaccess edge computing (MEC) platform was used as a middle layer to collect and process data from local devices. This work used deep neural networks as the detection models and used model-based transfer to achieve personalized models.

Since most IoT devices have low computation resources, most relevant works use middle layers (security gateways or MEC platforms) to launch the NN models and process data from IoT devices as hosts. This strategy might fail if the device disconnects from the servers [9], and the whole system would be at risk if any device was compromised by attackers. This article aims to discover a distributed anomaly-based IDS using ML models and deploy it on IoT/IIoT devices with federated learning for the model aggregation. Specifically, we adopt ensemble learning models such as AdaBoost and Random Forest (RF) on hardware platforms and operate an aggregation method by calculating accuracy score ranks and a weighted voting strategy for final output.

C. Transfer Learning

Transfer learning aims to transfer knowledge learnt from one or more existing domains to another domain [22]. Depending on the different types of knowledge to transfer, there are four transfer learning approaches: instance-based, feature-based, model-based, and relational-based transfer learning.

Transfer learning approaches can address data-imbalanced issues and reduce training cost by directly transferring the model or its parameters to new scenarios. Recent works on transfer learning focus on evaluating the difference between domains and tend to reduce the difference by instance reweighting and feature matching [11].

Recent studies have applied transfer learning to IoT intrusion detection and achieved significant progress. Alothman et al. [23] introduced an approach using instance-based transfer algorithms to solve data class-imbalanced problems for Botnet family classifications. Sun et al. [24] adopted the instance-based transfer learning algorithm TrAdaBoost [13] for multiclass network traffic classification to detect intrusions and abnormal behaviors, which simulate a typical scenario when a new threat occurs while the training data are limited. In this article, we adopt instance-based transfer for data reuse and apply this under a federated learning framework.

III. System Design

A. Overview

This work addresses imbalanced and non-IID training data concerns to deploy intrusion detection in IIoT devices with a privacy-preserved knowledge-sharing architecture. We adopt federated learning to improve the privacy of each device when sharing the model, and use transfer learning to boost the training of the models deployed on each device.

As shown in Fig. 1, the security cloud server (SCS) is a high-performance computing device which holds a large amount of public data and performs the model initialization and aggregation. The server has three tasks. First, it connects and manages all IoT clients. Second, it distributes initialized models to clients. Third, it collects updates from clients and processes model aggregations. The clients are a variety of separated IoT endpoints connected to the server. Sharing information between devices is not allowed, and hardware for each device could vary. The model from the server should be lightweight so it can fit the low-cost IIoT devices.

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 08:06:50 UTC from IEEE Xplore. Restrictions apply.

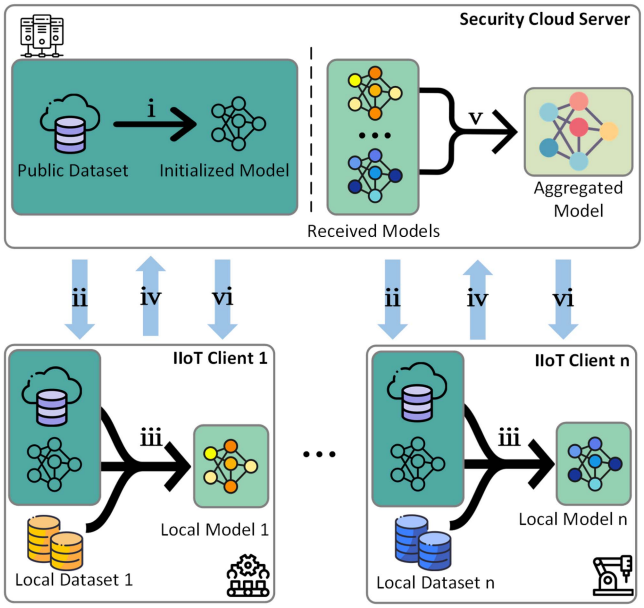

Fig. 1. Overview of the federated learning architecture for deploying anomaly-based intrusion detection models on IIoT devices. The server initialises the model based on the public dataset and distributes to IIoT clients. IIoT clients train the model at local and adopt instance-based transfer to reuse the public dataset as the localized data are non-IID and class imbalanced. Then IIoT clients return model updates to the server. The server aggregates received updates and returns the aggregated model to clients. Details of this figure are listed in Section III.A.

The training process has six main steps in the framework, as shown in Fig. 1. Details of each step are as follows.

i) The SCS initializes ML models based on the public data.
ii) The SCS distributes the public dataset and the model to all clients. Public data work as the source domain for transfer learning, and the model works as the baseline for local training.
iii) Each client uses the model received from step ii) as a base classifier in instance-based transfer learning.
iv) After completing the whole localized training process, clients return the updated classifiers back to the server.
v) The SCS can process model aggregation when it receives all updates from clients. We adopt average and rank strategies for model aggregation and propose the weighted voting algorithm for generating the final IDS engine.
vi) The SCS distributes the trained IDS engine back to clients for the next iteration.

B. Instance-Based Transfer Learning Mechanism

The proposed framework adopts a federated learning architecture for privacy-preserving ML and coordination between distributed devices. The SCS initializes a model based on public data and forwards it to clients. Clients then train this model based on localized data. As mentioned in Sections I and II, we adopt instance-based transfer on local machines to improve the on-device model training performance.

Data collected on devices would be class imbalanced and size constrained, which can significantly impact the overall performance. For federated learning, non-independent and identically distributed (non-IID) on-device datasets can also impair the coordination and knowledge sharing between clients. The TrAdaBoost [13] algorithm uses public data as the source domain and transfers it for training in the target. The motivation is to filter reusable instances by reducing sample weights for misclassified instances in the source domain and apply boosting algorithms to achieve better performance.

After receiving the initialized model f_0 and public data D^S, the client initializes the instance weight factors: $W^S = (w_1^S, w_2^S, \ldots, w_n^S)$ for $D_i^S = \{(x_i^S, y_i^S)\}, i = 1, 2, \ldots, n$ from the public data as the source, and sets up weights $W^T = (w_1^T, w_2^T, \ldots, w_m^T)$ for localized data D^T as the target domain, $D_j^T = \{(x_j^T, y_j^T)\}, j = 1, 2, \ldots, m$.

$$w_i^S = 1/n, \text{ for } i = 1, 2, \ldots, n \tag{1}$$

$$w_j^T = 1/m, \text{ for } j = 1, 2, \ldots, m. \tag{2}$$

Instances collected on the clients record features of network packets, and x are the packet features while y is the label. Then, the client tests the model f_0 using both source and target domains and returns predicted labels H_k.

$$H_k = \{h_1^S, h_2^S, \ldots, h_n^S, h_{n+1}^T, h_{n+2}^T, \ldots, h_{n+m}^T\}. \tag{3}$$

The weighted error rate ε_k stands for the total error rate for all misclassified instances

$$\varepsilon_k^t \doteq \sum_1^n \frac{w_j^T \left[y_j^T \neq h_j^T\left(\mathbf{x}_j^T\right)\right]}{\sum_j w_j^T}. \tag{4}$$

β_S and β_T are to adjust weights for instances in source and target domains. β_T is the weight factor for the target domain which is correlated to ε_k, while β_S is a static weight factor during each iteration for the source domain [25].

$$\beta_k^T \doteq \frac{1}{2} \ln \frac{1 - \varepsilon_k}{\varepsilon_k} \quad (\varepsilon_k < 1/2) \tag{5}$$

$$\beta^S \doteq \frac{1}{2} \ln \left(1 + \sqrt{2 \ln \frac{m}{K}}\right). \tag{6}$$

Then, adjusting source domain instance weights to highlight transferable data. When an instance is mistakenly classified, we use the weight factors w^S and w^T to reduce its impact in further training. The weight factors are updated based on the results H_t and iteration error rate ε_t.

$$w_i^{S_t} \leftarrow w_i^{S_{t-1}} e^{-\beta^S \left|y_i^S - h_i^{S_t}\right|}, \left|y_i^S - h_i^{S_t}\right| = 1 \text{ if } y_i^S \neq h_i^{S_t}$$
$$w_j^{T_t} \leftarrow w_j^T e^{\beta_k^T \left|y_j^T - h_j^{T_t}\right|}, \left|y_j^T - h_j^{T_t}\right| = 1 \text{ if } y_j^T \neq h_j^{T_t}. \tag{7}$$

As for the label settings, we use 0 for benign instances and 1 to p for the different attack methods such as DDoS, Botnet, etc. To test model performance on the test dataset D_{test}, we use predicted labels H_t from the last x iterations for predicted scores

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 08:06:50 UTC from IEEE Xplore. Restrictions apply.

pred(i).

$$\text{pred}(i) = \frac{\prod_{N-x}^{N} \beta_t^{T^{h_t(i)}}}{\left|\sup\left(\prod_{N-x}^{N} \beta_t^{T^{(H_t)}}\right) - \inf\left(\prod_{N-x}^{N} \beta_t^{T^{(H_t)}}\right)\right|} \quad \text{for } i = 1, 2, \cdots, s. \tag{8}$$

The TrAdaBoost algorithm in [13] works as a binary classifier and output hypothesis by comparing numeric values between $\prod_{t=N/2}^{N} \beta_t^{T^{-h_t(i)}}$ and $\prod_{t=N/2}^{N} \beta_t^{T^{-1/2}}$ while we use multiclass datasets to train and test. In this case, we present label hypothesis $h(i)$ depending on the range in which the pred(i) sits.

$$h(i) = \begin{cases} 0, \text{ for pred}(i) \leq 1/(1+p) \\ 1, \text{ for } 1/(1+p) < \text{pred}(i) \leq 2/(1+p) \\ \vdots \\ p, \text{ for } p/(1+p) \leq \text{pred}(i) \end{cases} \quad \text{for } i = 1, 2, \cdots, s. \tag{9}$$

C. Federated Learning and Model Aggregation

After receiving all generated models from clients, the server can perform aggregations to update the public model, and then return it to clients. One primary aggregation function in federated learning is FedAvg [12], which averages all received model parameters. In the proposed scenario, we use two ensemble ML methods as our base learners: 1) AdaBoost; and 2) RF. AdaBoost is a boosting algorithm which conjoins the output of multiple weak classifiers into a weighted sum for final results [13]. The RF method constructs many decision trees as its weak classifiers and uses bootstrap aggregating for ensemble learning to improve model accuracy and stability.

The FedAvg aggregation averages the weights and parameters of the base learners in model updates.

$$l(j) = \frac{\sum_{i=1}^{r} l_i(j)}{r}, \text{ for } j = 1, 2, \ldots, q \tag{10}$$

where $l(j)$ stands for base learners (we use decision trees (DT) in AdaBoost and RF models) in ensemble learning. r is the number of clients, and q is the number of base learners used in ML models.

Most works in federated learning adopt NNs [10] and researchers can proceed with aggregation mechanisms by finetuning model parameters for all or selected layers of the network. We use ensemble learning models in the proposed method, which involves multiple independent DT models. In rank aggregation, as presented in Fig. 2, the server tests the performance of all DT learners from updated models and selects DT models based on ranks of accuracy scores. Then, the server aggregates base learners into the aggregated model and distributes it to clients. This aggregation method can use different test data to evaluate the accuracy scores of base learners and aggregates personalized models for different network environments.

The rank aggregation uses accuracy scores to select base learners and assembles them as new IDS models. Each base learner usually has a different performance in detecting diverse attacks. For example, if the training dataset only has one specific attack class, the trained base learners would be more capable of detecting this attack class and less accurate with the others. In this case, rank aggregation can be introduced to customize the ensemble learning models based on different detection tasks. To boost the model performance, we present this weighted voting mechanism, as presented in Fig. 3, using the

Algorithm 1: On-Device Model Training Procedures.

Require: Public dataset: D^S;
Private datasets: D_i^T, $i = 1, 2, \ldots, r$;
Test dataset: D_{test};
Initialized model: Γ_0;
Ensure: Predicted labels for D_{test}: H

1: **TrAdaBoost:**
2: Initialize source and target sample weights W_0^S and W_0^T for D^S and D^T;
3: **for** k in N **do**
4: Use D^S and D^T to train model Γ_k with sample weights $W_{(k-1)}^S$ and $W_{(k-1)}^T$, return predicted labels H_k;
5: Calculate and record error rate ε_k and weight factors β^S, β_k^T by (4), (5) and (6);
6: Update sample weights W_k^S and W_k^T for instances in the source and target domains by (7);
7: **end for**
8: Evaluate model performance on the test dataset D_{test} to get prediction scores pred(i), $(i = 1, 2, \ldots, s)$ by (8);
9: Convert prediction scores to output $h(i)$ by (9);
10: **return**local trained model Γ_N;
11: Transmit local trained models Γ_i $(i = 1, 2, \ldots, r)$ to the security cloud server for federated learning model aggregations.

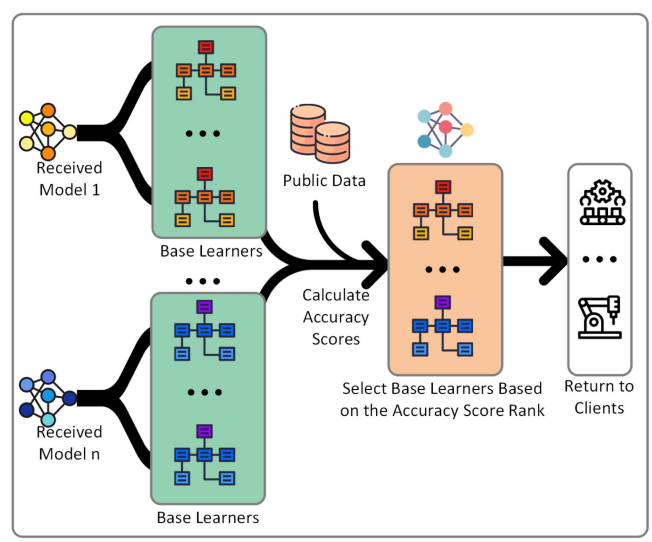

Fig. 2. Structure of the rank aggregation mechanism for ensemble learning models in federated learning. The server tests all base learners from updated ensemble learning models using the public test dataset. Then, the server creates a global aggregated model using base learners with the highest accuracy scores. Details of this aggregation mechanism are in Section III-C.

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 08:06:50 UTC from IEEE Xplore. Restrictions apply.

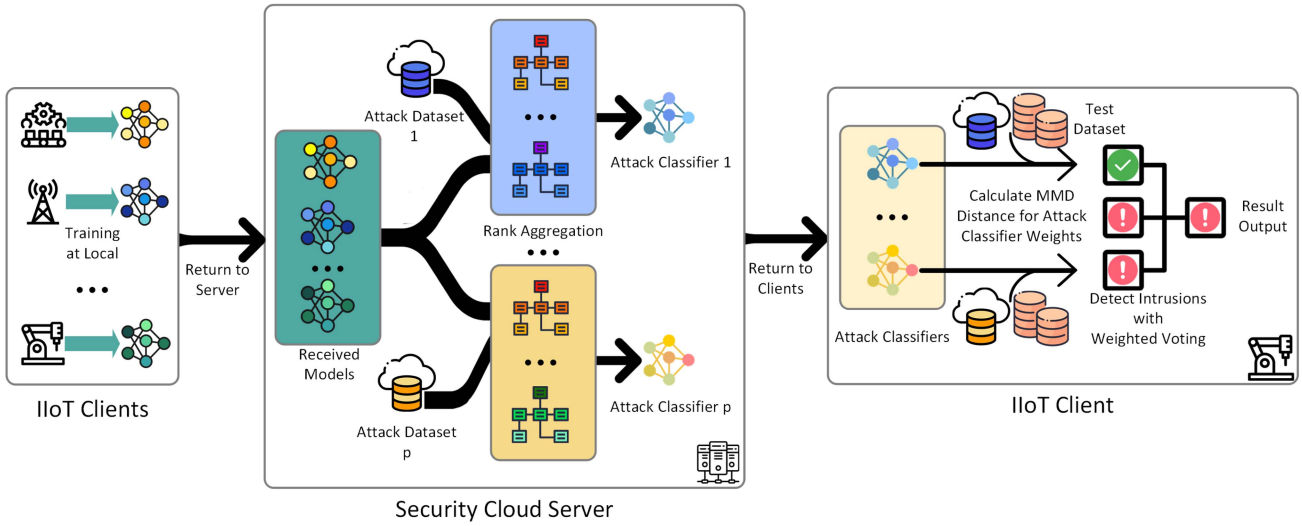

Fig. 3. Global view of the rank aggregation and the weighted voting mechanism. The server tests base learners using attack-specified datasets and aggregates classifiers for specific attacks. Then, the server returns attack-specified datasets and classifiers to all clients. To detect threats, clients can use all classifiers to get predicted labels and use MMD distance metrics to set up weights for classifiers. Finally, the IIoT client uses the weighted voting mechanism for the IDS engine. Details of this aggregation mechanism are in Section III-C.

Algorithm 2: Model Aggregation Strategies.

FedAvg Aggregation:
Average all attributes in received models $\Gamma_i,\ i = 1,\ 2, \ldots, r$ by (10);
Test on D_{test} and get predicted labels H;
return Γ_{avg}, H;
Rank Aggregation:
Achieve base learners $l(j),\ j = 1,\ 2, \ldots, q$ from received models $\Gamma_i,\ i = 1,\ 2, \ldots, r$;
Test base learners $l(j)$ accuracy on D_{test} and sort ranks;
Select q base learners with highest rank to get aggregated model Γ_{rank};
Test on D_{test} and get predicted labels H;
return Γ_{rank}, H;
Weighted Voting:
Generate attack classifiers γ_i based on binary class datasets $d_i,\ i = 1,\ 2, \ldots, p$ using the rank aggregation;
Calculate MMD distance and voting weights $wts(D_{\text{test}}, d_i),\ i = 1,\ 2, \ldots, p$ for attack classifiers;
Use all γ_i to test D_{test} and get predicted labels H using weighted voting;
return H;

rank aggregation method to generate attack-specified detection models and use weighted voting for the final output. First, we use binary datasets d_i $(i = 1,\ 2, \ldots, p)$ for each attack class randomly selected from the CIC-IDS datasets in Table II. After the server receives model updates from IoT clients, it adopts the rank aggregation method to generate attack-specified classifiers γ_i $(i = 1,\ 2, \ldots, p)$. The attack-specified classifiers would be highly confident in detecting one targeted attack class and we can use all attack classifiers for detecting threats with the weighted voting strategy comprehensively. To set the voting weight for each attack classifier, we calculate the distribution difference between the test dataset D_{test} and binary datasets d_i. Here, we use MMD [26] by calculating the distance between two datasets in the reproducing kernel Hilbert space $\mathcal{H}$

TABLE II
DATA DISTRIBUTIONS OF CIC-IDS 2017 AND CIC-IDS 2018

Dataset	Attack class	Total packets	Anomaly %
CIC-IDS 2017	DDoS	225,745	56.71
	Botnet	191,033	1.03
	PortScan	286,467	55.48
	Overall	**703,245**	**41.08**
CIC-IDS 2018	DDoS	1,048,574	57.39
	Botnet	1,048,575	17.76
	BruteForce	1,048,575	36.33
	Infiltration	331,100	28.11
	Overall	**3,476,824**	**39.17**

The bold entities highlighting the overall numbers in each dataset.

$$\text{MMD}[\mathcal{F}, n, m] := \sup_{f \in \mathcal{F}} \left(\frac{1}{n} \sum_{i=1}^{n} f(x_i) - \frac{1}{m} \sum_{i=1}^{m} f(y_i) \right) \tag{11}$$

where $\sup_{f \in \mathcal{F}}(\cdot)$ stands for supremum of the difference between the expectations of two datasets or instances. n and m are the number of samples in the two domains, and $f(\cdot)$ is the radial basis function (RBF) kernel in $\mathcal{H}$

$$f(\cdot) = \langle f, \phi(\cdot) \rangle_{\mathcal{H}}. \tag{12}$$

Here, we use the Gaussian kernel function for the RBF to calculate MMD distances

$$K(x, y) = \exp\left(\|x - y\|^2 / 2 \right). \tag{13}$$

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 08:06:50 UTC from IEEE Xplore. Restrictions apply.

The voting weights $\mathrm{wts}(X, Y)$ are the negative logarithm of corresponding MMD distances

$$\mathrm{wts}(X, Y) = -\log(2 * \mathrm{MMD}[\mathcal{F}, X, Y]) \tag{14}$$

where X is the test dataset D_{test} and Y stands for the binary datasets $d_i,\ (i = 1,\ 2, \ldots, p)$. We use the MMD weights for attack models $\gamma_i,\ (i = 1,\ 2, \ldots, p)$ and adopt the voting mechanism for final prediction results using attack models with weights.

D. Learning Procedures

The learning processes of localized model training are summarized in Algorithm 1. The learning processes of FedAvg, as well as the rank aggregation and weighted voting strategies for ensemble learning models are summarized in Algorithm 2.

IV. Experiment Setup

To evaluate the model performance, we established a physical test platform to simulate an IIoT network intrusion detection system. We selected two datasets, CIC-IDS 2017 and CIC-IDS 2018 [27] as the attack database. In the experiment, we implemented the federated learning model training process on multiple physical test platforms to simulate the IIoT setting.

A. Test Platform

This experiment simulates a scenario in which clients are connected from different industrial environments such as smart factories and smart grids supported by microcontrollers. The hardware and OS are different, as well as the local collected intrusion samples. Here, we use five Raspberry Pi® (three Pi3B™ and two Pi4B™) to present different IIoT hardware. All Pis are using ARM©Cortex-A™ series microprocessors and we use two Debian-based and three Arch-based Linux OS for Pis for different embedded systems. A high-performance desktop runs Ubuntu Linux 20.04 with Intel©Core™ i9-9900 K CPU@3.60 GHz and 64-GB memory, which works as the central server to connect all clients and perform model aggregation.

B. Datasets

This experiment simulates a scenario where localized datasets are privacy preserved, and the samples are limited. Meanwhile, the datasets are likely incomplete and only contain specific attacks. We use the CIC-IDS datasets [27] of network traffic, collected from the real-world network when intrusions occurred. As shown in Table II, we selected the three most common types of attack from CIC-IDS 2017[1] and four from CIC-IDS 2018[2] for model training and evaluation. In this article, we consider the following network threats and analyze the threat models as follows.

1) DDoS: DDoS attacks aim to deplete the victims' bandwidth or processing capacities through a flood of network traffic from multiple compromised devices as the source of attack traffic. Attackers can also launch reflective-based DDoS attacks by sending modified request messages to benign networked devices, and the spoofed devices return large response traffic to victims [28]. Networked IoT/IIoT devices can be vulnerable to DDoS attacks as they rely on critical network connections to transmit collected data and receive control instructions.
2) Botnet: Botnet attacks refer to malware that can infect targeted devices through system vulnerabilities, and hand over control to malicious actors. Attackers can use the botnet to accomplish intrusions, including ransomware, data-stealing, or launching DDoS attacks. There are a few types of Botnet malware, such as Mirai and Gafgyt [5], targeting IoT/IIoT devices such as web cameras, routers, and networked printers. Botnet malware can be imperceivable, and IoT/IIoT holders usually maintain devices at a low frequency, making the IoT/IIoT network a perfect place to breed bots for malicious activities.
3) PortScan: Port scanning is a common technique for attackers to search for open doors or vulnerabilities in a network. IoT/IIoT with embedded systems usually lack firewalls or security software against port scanning. Once attackers find open ports, they can send malicious packets or malware to break through the security barrier.
4) BruteForce: A brute-force attack uses a dictionary and tests all login passwords or encryption keys so the attacker can access the system without using any malware. Some networked IoT/IIoT users keep the factory settings or use weak passwords to log in [5], leading to a high success rate for brute-force attacks targeting IoT/IIoT devices.
5) Infiltration: Infiltration attacks usually exploit vulnerable applications to execute backdoor and can conduct different attacks targeting the victims. This type shows more diverse attack methodologies, making it challenging to identify.

TABLE III
Training and Test Data Distributions

Configurations	Attack types with percentage	Total instances
Public (D^S)	DDoS(20%), Botnet(20%), PortScan(20%), BruteForce(10%), Infiltration(10%)	10 000
Client1 (D_1^T)	DDoS(10%), Botnet(10%)	4400
Client2 (D_2^T)	DDoS(10%), Botnet(5%), BruteForce(10%), Infiltration(10%)	5000
Client3 (D_3^T)	DDoS(10%), Botnet(5%), PortScan(10%)	4500
Client4 (D_4^T)	PortScan(5%), Infiltration(10%), BruteForce(10%)	5760
Client5 (D_5^T)	BruteForce(10%), Infiltration(10%)	4000
Test (D_{test})	DDoS(15%), Botnet(15%), PortScan(15%), BruteForce(15%), Infiltration(15%)	7500

Table III presents the distributions of training data in the public dataset D^S and five on-device datasets D^T. D^S includes benign instances and all attack classes so that all clients can reuse data from it. Meanwhile, we set five different class-imbalanced and non-IID datasets as the target domains at local. The source domain consists of CIC-IDS 2017 and only a few samples for BruteForce and Infiltration attacks from 2018, while most instances in target domains are from CIC-IDS 2018. The test dataset D_{test} is class balanced so that the evaluation metrics can comprehensively present the model performance.

[1]https://www.unb.ca/cic/datasets/ids-2017.html
[2]https://registry.opendata.aws/cse-cic-ids2018/

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 08:06:50 UTC from IEEE Xplore. Restrictions apply.

C. Evaluation Settings

We use four metrics to evaluate the model performance. True-positive rate (TPR) calculates the rate for attack samples that are classified correctly. False-positive rate (FPR) calculates the rate for benign samples that are classified incorrectly. Accuracy (ACC) presents the rate for both attack and benign samples classified correctly

$$\mathrm{ACC} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}} \tag{15}$$

$$\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \tag{16}$$

$$\mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{TN} + \mathrm{FP}} \tag{17}$$

where TP is the true positive, standing for the number of attack samples correctly classified. FP is the false positive, standing for the number of benign samples incorrectly classified. TN is the true negative, standing for the number of benign samples correctly classified. FN is the false negative, standing for the number of attack samples incorrectly classified. Furthermore, to prevent misleading accuracy from imbalanced test data, we use the F_1 score for evaluating model performance

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{\mathrm{TP}}{\mathrm{TP} + \frac{1}{2}(\mathrm{FP} + \mathrm{FN})}. \tag{18}$$

The precision stands for the positive predictive value, presenting the percentage of TP in all positive samples $(\mathrm{TP} + \mathrm{FP})$. The recall presents the percentage of TP in all retrieved sets $(\mathrm{TP} + \mathrm{FN})$.

V. Experiment Results

A. Overview

This section presents the experimental results in cloud-based NN with two ensemble learning methods (AdaBoost and RF) and the support vector machine (SVM) as the classifiers to evaluate performance. We also implement a fully connected NN model using TensorFlow to simulate the centralized network intrusion detection for IIoT. The NN adopts two fully connected layers with 20 nodes each and a rectified linear unit as activation. The NN model uses categorical cross-entropy as the loss function, and we use the Adam optimizer for the optimization algorithm. Since the localized datasets are unreachable for cloud-based IDS, the NN uses public dataset D^S to train and test using test dataset D_{test}. The overall accuracy for the NN reaches 70.73% with F_1 score 0.661. The experiment comprises two major components: First, examining the improvements of instance-based transfer at local in training; and second, comparing the performance of model aggregation approaches with the weighted voting algorithm while utilizing ensemble ML techniques.

B. Performance Results for Localized Model Training

The results in Fig. 4 show the detection accuracy for five clients and FedAvg results in six different settings. When using the same ML models, accuracy scores with transfer are higher than the same models without transfer. This is because the on-device training datasets are imbalanced while the transfer mechanism can reuse instances from the public data, which significantly improves the performance. Furthermore, the detection accuracy without transfer is unstable because the local datasets are non-IID, which leads to biased models and impacts the aggregation process. Comparing the results in FedAvg, the transfer mechanism improves the accuracy by 25.64% in AdaBoost, 12.5% in SVM, and 16.05% in RF.

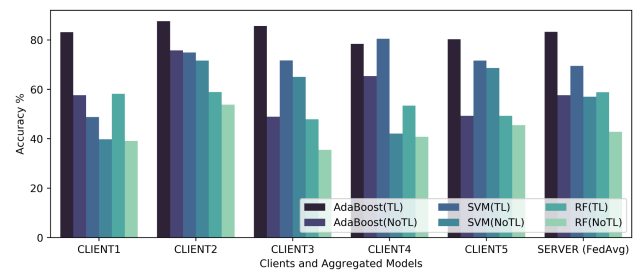

Fig. 4. Performance comparison of AdaBoost, RF, and SVM on five clients with or without transfer setting. The server uses FedAvg for model aggregation in different settings.

Fig. 5 shows the training process on each client in six different settings. Solid lines present the test accuracy on localized datasets (target domains) by iterations using different ML models, while dashed lines present the test accuracy on public datasets (source domains) by iterations using different ML models. All five figures show significant improvements compared with the initialized global model received from the server. Comprehensively, models tested by target domains achieve better results with iteration. In contrast, the training has a limited impact on the model performance in the source domain. This is because the training process is based on the target domains, each client trains a model based on the local data, and only a limited number of source domain instances are transferable.

The test platform is lightweight using Raspberry Pi® s to simulate real-world IIoT devices. When processing model training on Pis, we measure the resource consumption and hardware usage using three metrics: CPU usage, CPU frequency, and RAM usage. Fig. 6 shows the three metrics during training when using AdaBoost as the classifier and instance-based transfer for improvements. We use Pi4B™ for Clients 1 and 2, and Pi3B™ for Clients 3, 4, and 5. Compared with no-load status, the overall CPU consumption is usually less than 25% for model training. RAM usage results in Fig. 6 clearly show two phases in the experiment: 1) initialized status with socket connections set up; and 2) localized model training. On average, it takes approximately 100 MB of memory to initialize a client and connect to the server on Pis. The model training process of TrAdaBoost consumes approximately 50 MB of memory. Furthermore, the weighted voting algorithm can process 7500 recorded packets in 28.57 s with 95.97% overall accuracy when using AdaBoost. We believe the proposed framework is relatively lightweight for resource-constrained IIoT and mobile devices and has practical availability in real-world scenarios for IIoT intrusion detection.

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 08:06:50 UTC from IEEE Xplore. Restrictions apply.

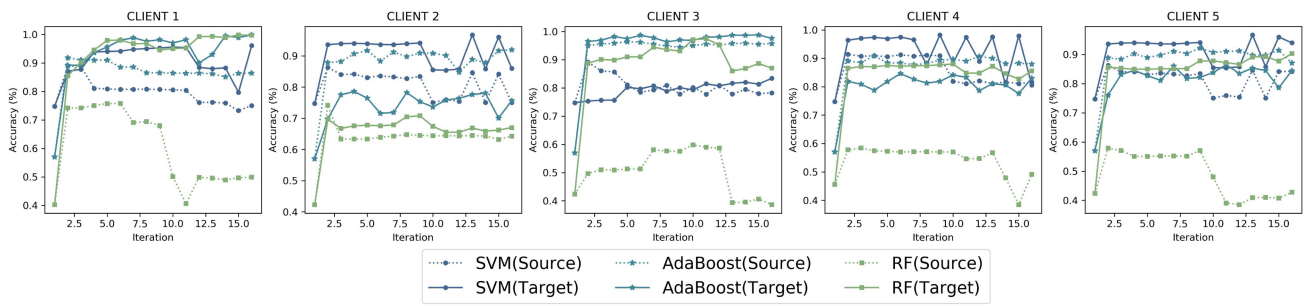

Fig. 5. Accuracy scores for the source and target domains on clients using SVM, AdaBoost, and RF.

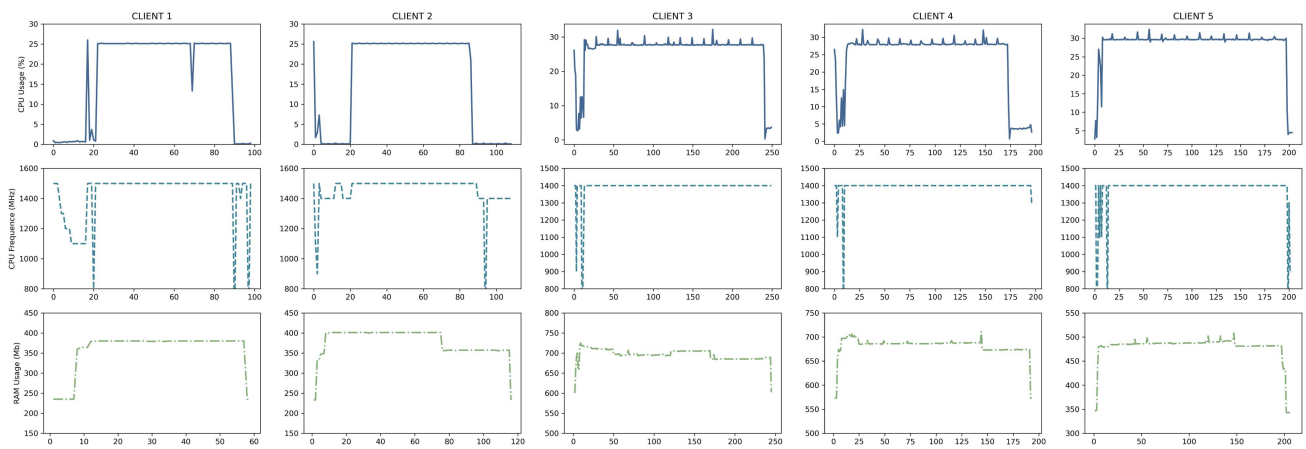

Fig. 6. This figure records three metrics (CPU usage, CPU frequency, and RAM usage) during the model training process on five Raspberry Pis to present the resource consumption status on the real-world test platform.

TABLE IV
ACCURACY SCORES BY ATTACK CLASSES FOR ATTACK-SPECIFIED CLASSIFIERS IN ADABOOST AND RF

	DDoS	Botnet	PortScan	BruteForce	Infiltration
AdaBoost(DDoS)	**0.987**	0.411	0.269	0.391	0.415
AdaBoost(Botnet)	0.191	**0.992**	0.393	0.507	0.541
AdaBoost(PortScan)	0.198	0.599	**0.995**	0.475	0.370
AdaBoost(BruteForce)	0.147	0.331	0.234	**0.998**	0.588
AdaBoost(Infiltration)	0.203	0.611	0.400	0.585	**0.659**
RF(DDoS)	**0.978**	0.508	0.331	0.450	0.248
RF(Botnet)	0.196	**0.991**	0.390	0.587	0.591
RF(PortScan)	0.200	0.597	**0.997**	0.599	0.566
RF(BruteForce)	0.199	0.593	0.397	**0.999**	0.570
RF(Infiltration)	0.197	0.587	0.394	0.599	**0.679**

The bold entities highlighting the highest accuracy scores on each row.

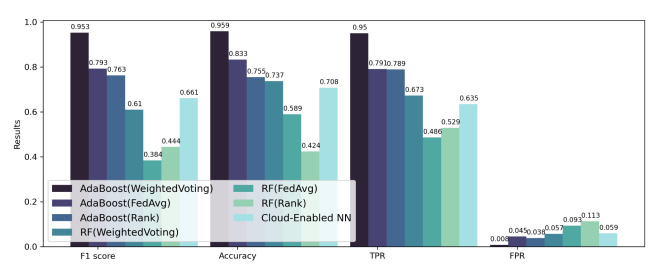

Fig. 7. Model evaluation metrics for aggregated models and the cloud-based NN.

C. Performance Results for Model Aggregations

Table IV presents the accuracy scores of attack-specified AdaBoost and RF classifiers aggregated by the rank aggregation method. The results show classifiers reach the highest performance on one specific attack type while acting poorly on others. The weighted voting method presented in Fig. 3 aims to use these classifiers for targeted attacks and maximize overall accuracy scores. Table V presents the accuracy scores of each attack class to evaluate model aggregation performance using FedAvg and the weighted voting method. This table also provides accuracy scores of the models (NN, AdaBoost, and RF) trained in a centralized way on the cloud to evaluate the federated learning performance. The results show the performance of federated learning with the weighted voting method can reach the same level as cloud-based models, and the proposed method can achieve higher accuracy for some specific attack classes.

Fig. 7 compares performance of NN, FedAvg, and Rank aggregation, as well as the weighted voting algorithm on AdaBoost and RF in four metrics: F_1 scores, accuracy (ACC), true-positive rate (TPR), and false-positive rate (FPR).

As for a baseline, accuracy of the cloud-enabled NN reaches 70.73% with 0.635 as its TPR and 0.059 as its FPR. Comparing two aggregation methods in AdaBoost, the F_1 scores for FedAvg reaches 0.793 and the score for rank aggregation researches 0.763. The FPRs are very close in the two aggregated models (0.037% for the FedAvg and 0.038% for aggregation with rank). Comparing the aggregation results in RF, the FedAvg shows

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 08:06:50 UTC from IEEE Xplore. Restrictions apply.

TABLE V
ACCURACY SCORES BY ATTACK CLASSES FOR AGGREGATED MODELS, COMPARED WITH CENTRALIZED MODELS TRAINED ON THE CLOUD

	DDoS	Botnet	PortScan	BruteForce	Infiltration	Overall
NN (centralized)	0.694	0.490	0.450	0.758	0.619	**0.661**
AdaBoost (entralized)	0.795	0.982	0.999	0.999	0.938	**0.961**
AdaBoost (weighted voting)	0.803	0.991	0.998	0.999	0.938	**0.959**
AdaBoost (FedAvg)	0.960	0.974	0.985	0.788	0.601	**0.833**
RF (entralized)	0.510	0.921	0.941	0.772	0.517	**0.774**
RF (weighted voting)	0.463	0.985	0.992	0.757	0.603	**0.737**
RF (FedAvg)	0.801	0.510	0.898	0.600	0.569	**0.589**

The bold entities highlighting the overall accuracy scores.

higher accuracy (58.90%) than the rank aggregation (42.36%). In contrast, the model aggregated using the rank algorithm can better classify threats. Overall, the weighted voting algorithm significantly improves the IDS performance, and the accuracy scores reach 95.97% for AdaBoost and 73.7% for RF. Comparing the F_1 scores between the proposed algorithm and FedAvg, the results improve by approximately 0.160 in AdaBoost and 0.226 in RF. And the accuracy of the proposed algorithm outperforms the NN by 25.10% and 2.89% in AdaBoost and RF, respectively. To sum up, the rank aggregation can reach comparable performance as the FedAvg, and the weighted voting algorithm can significantly improve the performance as a novel approach for ensemble ML in a federated learning setting.

VI. DISCUSSIONS

Despite the experiment results showing that the instance-based transfer learning and proposed aggregation methods can significantly improve detection performance, there are still a few limitations and open questions for future work.

Though the instance-based transfer learning can significantly boost the training process at local, accuracy scores in Fig. 5 show fluctuation during training. This may be because in the experiment setting, training data distributions are non-IID and class imbalanced, leading to varied results in transfer learning. As part of our future work, we plan to discover instance- or model-based transfer techniques for federated learning to achieve model personalizations and coordinate model training processes for non-IID data or heterogeneous scenarios.

Second, we implement cloud-based AdaBoost, RF, and a NN for centralized intrusion detection models for comparisons. Results in Table IV show our work outperforms the centralized models when detecting some specific attack types and the metrics presented in Fig. 7 demonstrate that our approach outperforms the NN, supporting the effectiveness of this approach. However, we only test the aggregation method using ensemble ML models as base learners. The scalabilities of this work have not yet been discussed. We expect further actions to extend our work for more application scenarios.

Furthermore, the proposed method assumes that all devices can receive and return updates without network latency and outage. However, this work fails to discuss the asynchronous scenario or when the network is dynamic or error-prone. Model synchronizations during training would be a critical challenge for flexible devices such as mobile phones or unmanned vehicles. Novel security techniques are yet to be discovered for dynamic wireless network communication scenarios. Managing the federated learning model training process and optimizing the performance of all participants in an asynchronous scheme is one important and promising future work direction.

VII. CONCLUSION

This article aims to address critical security challenges in IIoT and proposed a lightweight anomaly-based intrusion detection approach using ensemble ML as classifiers. The proposed method adopted instance-based transfer learning to resolve training data constraints and federated learning for a privacy-preserved knowledge-sharing architecture. Furthermore, we introduced rank aggregation for the ensemble learning model (AdaBoost and RF) updates in federated learning. We also presented a weighted voting method to reuse base learners in updated models by generating attack-specified classifiers and using MMD distance scores to set up weights for each classifier. The proposed method showed significant improvements compared with the average mechanism and the rank aggregation. We established a physical test platform using lightweight hardware to simulate real-world IIoT scenarios for evaluations and presented comprehensive results to demonstrate the effectiveness and validity of our work.

REFERENCES

[1] M. A. Dootio, A. Lakhan, A. H. Sodhro, T. M. Groenli, N. Z. Bawany, and S. Kumar, "Secure and failure hybrid delay enabled a lightweight RPC and SHDS schemes in industry 4.0 aware IIoHT enabled fog computing," *Math. Biosci. Eng.*, vol. 19, no. 1, pp. 513–536, 2022. [Online]. Available: https://www.diva-portal.org/smash/record.jsf?pid=diva2:1615653

[2] Y. Fan, Y. Li, M. Zhan, H. Cui, and Y. Zhang, "IoTDefender: A federated transfer learning intrusion detection framework for 5G IoT," in *Proc. IEEE 14th Int. Conf. Big Data Sci. Eng.*, 2020, pp. 88–95.

[3] M. Borhani, M. Liyanage, A. H. Sodhro, P. Kumar, A. D. Jurcut, and A. Gurtov, "Secure and resilient communications in the industrial internet," *Guide to Disaster-Resilient Communication Networks.* Cham, Switzerland: Springer, 2020, pp. 219–242.

[4] J. Arshad, M. A. Azad, M. M. Abdeltaif, and K. Salah, "An intrusion detection framework for energy constrained IoT devices," *Mech. Syst. Signal Process.*, vol. 136, 2020, Art. no. 106436. [Online]. Available: https://doi.org/10.1016/j.ymssp.2019.106436

[5] NSFocus Information Technology Co Ltd, "2020 cybersecurity insights," Accessed: Nov. 1, 2022. [Online]. Available: https://nsfocusglobal.com/wp-content/uploads/2021/06/2020-NSFOCUS-Cybersecurity-Insights.pdf

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 08:06:50 UTC from IEEE Xplore. Restrictions apply.

[6] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Commun. Surv. Tut.*, vol. 18, no. 2, pp. 1153–1176, Apr.–Jun. 2016.

[7] C. Nykvist, M. Larsson, A. H. Sodhro, and A. Gurtov, "A lightweight portable intrusion detection communication system for auditing applications," *Int. J. Commun. Syst.*, vol. 33, no. 7, May 2020, Art. no. e4327.

[8] I. Ahmad et al., "Machine learning meets communication networks: Current trends and future challenges," *IEEE Access*, vol. 8, pp. 223418–223460, 2020. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9274307/

[9] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A. R. Sadeghi, "DÏoT: A federated self-learning anomaly detection system for IoT," in *Proc. Int. Conf. Distrib. Comput. Syst.*, 2019, vol. 2019-Jul., pp. 756–767.

[10] S. A. Rahman, H. Tout, C. Talhi, and A. Mourad, "Internet of things intrusion detection: Centralized, on-device, or federated learning?," *IEEE Netw.*, vol. 34, no. 6, pp. 310–317, Nov./Dec. 2020.

[11] Y. Chen, X. Qin, J. Wang, C. Yu, and W. Gao, "FedHealth: A federated transfer learning framework for wearable healthcare," *IEEE Intell. Syst.*, vol. 35, no. 4, pp. 83–93, Jul./Aug. 2020.

[12] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," pp. 1–10, 2016. [Online]. Available: http://arxiv.org/abs/1610.05492

[13] W. Dai, Q. Yang, G. R. Xue, and Y. Yu, "Boosting for transfer learning," *ACM Int. Conf. Proc. Ser.*, vol. 227, pp. 193–200, 2007.

[14] J. Sengupta, S. Ruj, and S. Das Bit, "A comprehensive survey on attacks, security issues and blockchain solutions for IoT and IIoT," *J. Netw. Comput. Appl.*, vol. 149, no. Sep. 2019, 2020, Art. no. 102481. [Online]. Available: https://doi.org/10.1016/j.jnca.2019.102481

[15] X. Yu and H. Guo, "A survey on IIoT security," in *Proc. IEEE VTS Asia Pacific Wireless Commun. Symp.*, 2019, pp. 1–5.

[16] Y. Hu, A. Yang, H. Li, Y. Sun, and L. Sun, "A survey of intrusion detection on industrial control systems," *Int. J. Distrib. Sensor Netw.*, vol. 14, no. 8, 2018, Art. no. 1550147718794615.

[17] M. Caselli, E. Zambon, and F. Kargl, "Sequence-aware intrusion detection in industrial control systems," in *Proc. 1st ACM Workshop Cyber- Phys. Syst. Secur., Part of ASIACCS*, pp. 13–24, 2015.

[18] M. Eskandari, Z. H. Janjua, M. Vecchio, and F. Antonelli, "Passban IDS: An intelligent anomaly-based intrusion detection system for IoT edge devices," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 6882–6897, Aug. 2020.

[19] H. Bostani and M. Sheikhan, "Hybrid of anomaly-based and specification-based IDS for Internet of Things using unsupervised OPF based on MapReduce approach," *Comput. Commun.*, vol. 98, pp. 52–71, 2017. [Online]. Available: http://dx.doi.org/10.1016/j.comcom.2016.12.001

[20] T. D. Nguyen, P. Rieger, M. Miettinen, and A.-R. Sadeghi, "Poisoning attacks on federated learning-based IoT intrusion detection system," Aug. 16, 2021. [Online]. Available: https://doi.org/10.14722/diss.2020.23003

[21] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, 2019.

[22] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[23] B. Alothman, H. Janicke, and S. Y. Yerima, "Class balanced similarity-based instance transfer learning for botnet family classification," *Lecture Notes Comput. Sci. (Including Subseries Lecture Notes Artif. Intell. Lecture Notes Bioinf.)*, vol. 11198 LNAI, pp. 99–113, 2018.

[24] G. Sun, L. Liang, T. Chen, F. Xiao, and F. Lang, "Network traffic classification based on transfer learning," *Comput. Elect. Eng.*, vol. 69, pp. 920–927, 2018. [Online]. Available: https://doi.org/10.1016/j.compeleceng.2018.03.005

[25] Y. Yao and G. Doretto, "Boosting for transfer learning with multiple sources," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2010, pp. 1855–1862.

[26] K. M. Borgwardt, A. Gretton, M. J. Rasch, H.-P. Kriegel, B. Schölkopf, and A. J. Smola, "Integrating structured biological data by kernel maximum mean discrepancy," *Bioinformatics*, vol. 22, no. 14, pp. e49–e57, 07 2006. [Online]. Available: https://doi.org/10.1093/bioinformatics/btl242

[27] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," *ICISSp*, vol. 1, pp. 108–116, 2018.

[28] X. Liu, L. Zheng, S. Helal, W. Zhang, C. Jia, and J. Zhou, "A broad learning-based comprehensive defence against SSDP reflection attacks in IoTs," *Digit. Commun. Netw.*, 2022. [Online]. Available: https://doi.org/10.1016/j.dcan.2022.02.008

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 08:06:50 UTC from IEEE Xplore. Restrictions apply.